

AI Components

This report outlines the Artificial Intelligence components and features integrated into the project, primarily focusing on the automated voice-to-ticket pipeline.

AI Components Used

- Speech-to-Text Engine
- Natural Language Processing (NLP) Engine

Primary AI Features

- **Multilingual Audio Transcription & Translation**
- **Intelligent Information Extraction**
- **Automated Categorization & Prioritization**
- **Sentiment Analysis**
- **Contextual Error Correction**

AI Models

1. **OpenAI Whisper (small model)**
 - **Purpose:** Audio transcription and translation (task="translate").
 - **Deployment:** Local execution via the whisper Python package.
2. **Meta Llama 3 (llama3)**
 - **Purpose:** NLP, JSON structured data extraction, and sentiment analysis.
 - **Deployment:** Local execution via the ollama engine.

Benefits

- Privacy and Data Security
- Cost Efficiency
- Streamlined Workflows
- Language Inclusivity

Limitations

- Hardware Dependent
- Hallucinations or Misinterpretations
- Model Size Constraints

Future AI Enhancements

- Real-Time Streaming
- Model Fine-Tuning
- Dynamic Model Selection
- Voice Synthesis
- Agent Assist Suggestions

Prompt Used

"Act as an Expert Full-Stack AI Engineer and UI/UX Designer. I want to build a bilingual 'Voice-to-Ticket' Customer Support automation web application called 'Ignite Ticket'.

1. Core Architecture:

Build the backend using Python and Flask.

Use a local SQLite database to store ticket information (ID, Title, Description, Category, Department, Priority, Sentiment, Audio File Path, and Key Details).

2. The AI Pipeline (Bilingual NLP):

When a user records or uploads audio, process it using the local OpenAI Whisper ('small') model. Configure it with task="translate" so it natively understands both spoken English and Tamil, and automatically translates/transcribes everything into standard English text.

Pass that English text to a local Llama 3 model (via Ollama). Give Llama 3 a strict system prompt instructing it to fix phonetic spelling errors (common in translated Tamil names) and output ONLY a structured JSON object containing: a polished Title and Description, Category, Department, Priority, Sentiment, and a nested object of Customer Details (Name, Email, Phone).

3. Automated Features:

Build an `email_service.py` using Python's `smtplib`. Every time the AI generates a new ticket, it should automatically send a beautifully formatted email alert containing all ticket details to a hardcoded admin email address.

4. Premium Frontend UI/UX (Glassmorphism):

Build a single-page Vanilla HTML/CSS/JS dashboard. Do not use Tailwind; write custom CSS.

The design must be a Premium White Glassmorphism theme. Use translucent panels (`backdrop-filter: blur`), soft pill-like rounded corners (12px to 24px), deep 3D box-shadows with inset highlights, and the Google 'Outfit' font.

Features: Include a dynamic audio recording ring that pulses when listening. Include a custom-built, glass-styled dropdown menu to filter tickets by department, and a glass-styled search bar that filters the ticket table dynamically as the user types.

Sub-pages: Include a `/history` dashboard to show analytics and resolved ticket counts, and a specific `/ticket/<id>` page to view deep details and playback the original call audio. Ensure there are functioning 'Mark as Resolved' and 'Delete' buttons."